Université de Perpignan
C. Legrand

L3 Informatique
Année 2026

Corrigé – TD2 Sécurité

Techniques d'infection et polymorphisme

Avertissement éthique et légal

Les techniques présentées dans ce TD sont étudiées dans un **cadre strictement académique** afin de comprendre les mécanismes utilisés par les logiciels malveillants et ainsi pouvoir mieux s'en protéger.
Toute utilisation de ces techniques en dehors du cadre pédagogique est illégale et passible de sanctions pénales (chapitre III du Code pénal, articles 323-1 à 323-8 – *Des atteintes aux systèmes de traitement automatisé de données*) :

- Accès ou maintien frauduleux dans un STAD (art. 323-1) : **3 ans** d'emprisonnement et **100 000 €** d'amende
- Entrave au fonctionnement d'un STAD (art. 323-2) : **5 ans** d'emprisonnement et **150 000 €** d'amende
- Introduction frauduleuse de données dans un STAD (art. 323-3) : **5 ans** d'emprisonnement et **150 000 €** d'amende

Peines portées à **7 ans** et **300 000 €** lorsque le système visé traite des données à caractère personnel mis en œuvre par l'État, et jusqu'à **10 ans** et **300 000 €** en bande organisée (art. 323-4-1). *Peines mises à jour par la loi LOPMI n° 2023-22 du 24 janvier 2023.*

Exercice 1 (Bombe logique avec déclencheur temporel)**.** Une *bombe logique* est un programme malveillant qui s'installe dans un système et attend un événement spécifique appelé **trigger** (déclencheur) pour exécuter sa fonction offensive (la **charge utile** ou *payload*).

On considère le programme C suivant :

```c
#include <stdio.h>
#include <time.h>

int check_trigger() {
    time_t now = time(NULL);
    struct tm *t = localtime(&now);

    // Condition de declenchement
    if (t->tm_mon == 3 && t->tm_mday == 1) {
        return 1;
    }
    return 0;
}

void payload() {
```

```c
    printf("=== PAYLOAD ACTIVATED ===\n");
    printf("All your files have been encrypted!\n");
}

void normal_operation() {
    printf("System running normally...\n");
}

int main() {
    if (check_trigger()) {
        payload();
    } else {
        normal_operation();
    }
    return 0;
}
```

1. Identifier le *trigger* utilisé dans ce programme et expliquer précisément son fonctionnement. À quelle date la charge utile sera-t-elle activée ?

2. Modifier la fonction `check_trigger()` pour que le déclenchement se produise uniquement si le programme a été exécuté plus de 100 fois.

3. Proposer deux techniques pour rendre le *trigger* plus difficile à détecter lors d'une analyse statique du code.

4. Du point de vue d'un antivirus, quelles techniques pourraient être utilisées pour détecter ce type de programme malveillant ?

Rappel de cours

Bombe logique : programme malveillant qui reste inactif (dormant) jusqu'à ce qu'une condition spécifique soit remplie. À la différence d'un virus, une bombe logique ne se reproduit pas : elle attend son *trigger* puis exécute sa *payload*.

Types de déclencheurs courants :

- **Temporel :** date ou heure précise (ex : le 1er avril, vendredi 13)
- **Compteur :** après N exécutions ou N jours d'utilisation
- **Événementiel :** suppression d'un compte utilisateur, absence de connexion pendant N jours
- **Réseau :** réception d'un paquet spécifique, contact avec un serveur C&C
- **Environnemental :** présence/absence d'un fichier, d'un processus, d'une clé de registre

Exemples historiques :

- **Omega Engineering (1996)** – Timothy Lloyd, administrateur système licencié, avait placé une bombe logique qui a supprimé les programmes de conception et de production de l'entreprise 20 jours après son départ. Dommages estimés : 10 millions de dollars.
- **Dark Seoul (2013)** – Attaque contre des banques et médias sud-coréens. Le malware attendait une date précise (20 mars 2013 à 14h00) pour effacer les disques durs de milliers de machines simultanément.

Correction 1. *1.* ***Identification du trigger :***

Le trigger est de type ***temporel****. Il utilise les fonctions* `time()` *et* `localtime()` *pour obtenir la date courante, puis vérifie deux conditions :*

- `t->tm_mon == 3` *: le mois est égal à 3*
- `t->tm_mday == 1` *: le jour du mois est égal à 1*

Attention

Dans la structure `struct tm` *du langage C, le champ* `tm_mon` *est indexé* ***à partir de 0*** *(janvier = 0, février = 1, . . . , décembre = 11). C'est un piège classique ! Donc* `tm_mon == 3` *correspond au mois d'****avril*** *(et non mars). La charge utile sera activée le* ***1er avril*** *(poisson d'avril !).*

Tableau complet des champs de `struct tm` ***:***

Champ	*Signification*	*Plage*	*Remarque*
`tm_sec`	*Secondes*	*0–60*	*60 pour seconde intercalaire*
`tm_min`	*Minutes*	*0–59*	
`tm_hour`	*Heures*	*0–23*	
`tm_mday`	*Jour du mois*	*1–31*	*Commence à 1*
`tm_mon`	*Mois*	*0–11*	***0 = janvier***
`tm_year`	*Année*	*depuis 1900*	***Ajouter 1900***
`tm_wday`	*Jour de la semaine*	*0–6*	*0 = dimanche*
`tm_yday`	*Jour de l'année*	*0–365*	*0 = 1er janvier*
`tm_isdst`	*Heure d'été*	*>0, 0, <0*	*Positif = heure d'été*

2. ***Modification pour compteur d'exécutions :***

```c
int check_trigger() {
    int count = 0;
    FILE *f = fopen("/tmp/.counter", "r");
    if (f) {
        fscanf(f, "%d", &count);
        fclose(f);
    }

    count++;

    f = fopen("/tmp/.counter", "w");
    if (f) {
        fprintf(f, "%d", count);
        fclose(f);
    }

    return (count > 100) ? 1 : 0;
}
```

Remarques :

- *Le fichier* `/tmp/.counter` *stocke le compteur de manière persistante*
- *Le préfixe "." rend le fichier caché sous Linux*
- *Le chemin* `/tmp/` *est accessible en écriture par tous les utilisateurs*

3. ***Techniques d'obfuscation du trigger :***

Technique 1 : Chiffrement des constantes

Au lieu d'écrire directement les valeurs 3 et 1, on peut les stocker chiffrées et les déchiffrer à l'exécution :

```c
int month_trigger = 0x41 ^ 0x42;  // 3, mais pas visible directement
int day_trigger = 0x43 ^ 0x42;    // 1, mais pas visible directement
```

Technique 2 : Calculs indirects

Utiliser des expressions arithmétiques pour masquer les valeurs :

```c
if (t->tm_mon == (15 / 5) && t->tm_mday == (10 - 9))
```

Autres techniques possibles :

- *Stocker la date cible dans un fichier de configuration externe*
- *Utiliser des variables d'environnement*
- *Récupérer la date depuis un serveur distant*

4. ***Techniques de détection antivirus :***

   - ***Analyse comportementale :*** *Surveiller les appels aux fonctions* `time()`, `localtime()` *suivis d'un branchement conditionnel*
   - ***Détection heuristique :*** *Identifier les patterns de "code dormant" avec des conditions temporelles*
   - ***Sandboxing avec manipulation du temps :*** *Exécuter le programme dans un environnement virtuel en accélérant le temps système pour déclencher le payload*
   - ***Analyse statique :*** *Rechercher des comparaisons sur les champs de* `struct tm`
   - ***Émulation :*** *Simuler l'exécution du programme avec différentes dates*

Point clé à retenir

- *Une* ***bombe logique*** *est un programme dormant qui attend un événement (trigger) pour exécuter sa charge utile (payload).*
- *Le champ* `tm_mon` *de la structure* `struct tm` *est indexé à partir de* ***0*** *(piège classique en C).*
- *L'obfuscation des constantes (XOR, calculs indirects) complique l'analyse statique mais n'empêche pas l'analyse dynamique.*
- *Les antivirus modernes combinent plusieurs techniques (statique, dynamique, heuristique) pour détecter les bombes logiques.*

Exercice 2 (Chiffrement XOR et polymorphisme)**.** Le **polymorphisme** est une technique utilisée par les virus pour échapper à la détection par signature. En chiffrant le code viral avec des clés différentes à chaque copie, le virus produit des signatures binaires différentes tout en conservant le même comportement.

Le chiffrement **XOR** (ou exclusif) est souvent utilisé car il possède une propriété intéressante : appliquer deux fois la même opération avec la même clé redonne la valeur originale.

On considère le programme C suivant :

```c
#include <stdio.h>
#include <string.h>

void xor_crypt(char *data, int len, char key) {
    for (int i = 0; i < len; i++) {
        data[i] = data[i] ^ key;
    }
}

int main() {
    char payload[] = "FORMAT C: /Y";
```

```c
    char key = 0x42;

    printf("Original : %s\n", payload);

    // Chiffrement
    xor_crypt(payload, strlen(payload), key);
    printf("Chiffre  : ");
    for (int i = 0; i < strlen(payload); i++) {
        printf("%02X ", (unsigned char)payload[i]);
    }
    printf("\n");

    // Dechiffrement
    xor_crypt(payload, strlen(payload), key);
    printf("Dechiffre: %s\n", payload);

    return 0;
}
```

1. Expliquer mathématiquement pourquoi l'opération XOR permet à la fois de chiffrer et de déchiffrer avec la même fonction.

2. Exécuter mentalement le programme avec la clé `0x42`. Quel est le premier octet du payload chiffré ? (Le code ASCII de 'F' est 0x46)

3. Modifier la fonction `xor_crypt()` pour qu'elle utilise une clé multi-octets (par exemple `"SECRET"`) au lieu d'un seul octet.

4. Du point de vue d'un antivirus :

   (a) Pourquoi cette technique rend-elle la détection par signature inefficace ?

   (b) Quelle technique alternative l'antivirus pourrait-il utiliser pour détecter ce type de code polymorphe ?

Rappel de cours

L'opération XOR (ou exclusif) est une opération logique bit à bit notée $\oplus$. Elle retourne 1 si et seulement si les deux bits en entrée sont différents.

Table de vérité :

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Polymorphisme viral : technique consistant à chiffrer le corps du virus avec une clé différente à chaque copie. Seule la routine de déchiffrement reste en clair. Ainsi, chaque copie du virus a une signature binaire différente, rendant la détection par signature impossible.

Lien avec le chiffrement de Vernam (One-Time Pad) : Le XOR est au cœur du chiffrement de Vernam, le seul système cryptographique prouvé **théoriquement incassable** (Shannon, 1949). Conditions nécessaires : la clé doit être (1) aussi longue que le message, (2) parfaitement aléatoire, et (3) utilisée une seule fois. En pratique, les virus réutilisent des clés courtes, ce qui rend le chiffrement vulnérable à l'analyse fréquentielle.

Correction 2. 1. ***Explication mathématique du XOR :***

L'opération XOR ($\oplus$) possède les propriétés suivantes :

- ***Associativité :*** $(A \oplus B) \oplus C = A \oplus (B \oplus C)$
- ***Commutativité :*** $A \oplus B = B \oplus A$
- ***Élément neutre :*** $A \oplus 0 = A$
- ***Auto-inverse :*** $A \oplus A = 0$ *(tout élément est son propre inverse)*

Démonstration : $A \oplus K \oplus K = A \oplus (K \oplus K) = A \oplus 0 = A$

Ainsi, si $C = A \oplus K$ (chiffrement), alors $C \oplus K = A \oplus K \oplus K = A$ (déchiffrement).

La même opération avec la même clé permet donc de chiffrer et déchiffrer.

2. ***Calcul des octets chiffrés :***

Premier octet :

- *Caractère 'F' en hexadécimal :* `0x46` = `01000110` *en binaire*
- *Clé* `0x42` = `01000010` *en binaire*

Calcul XOR bit à bit :

```
  01000110   (0x46 = 'F')
^ 01000010   (0x42 = cle)
----------
  00000100   (0x04)
```

Le premier octet chiffré est ***0x04*** *(caractère de contrôle EOT – End Of Transmission).*

Tableau complet du chiffrement XOR de "FORMAT C: /Y" avec la clé 0x42 :

Caractère	*ASCII (hex)*	*Clé (hex)*	*XOR (hex)*	*XOR (décimal)*
F	0x46	0x42	0x04	4
O	0x4F	0x42	0x0D	13
R	0x52	0x42	0x10	16
M	0x4D	0x42	0x0F	15
A	0x41	0x42	0x03	3
T	0x54	0x42	0x16	22
(espace)	0x20	0x42	0x62	98
C	0x43	0x42	0x01	1
:	0x3A	0x42	0x78	120
(espace)	0x20	0x42	0x62	98
/	0x2F	0x42	0x6D	109
Y	0x59	0x42	0x1B	27

Résultat chiffré complet : `04 0D 10 0F 03 16 62 01 78 62 6D 1B`

Attention

Attention aux erreurs de calcul XOR fréquentes : il faut opérer bit par bit et non octet par octet. Par exemple, `0x20` $\oplus$ `0x42` $=$ `0x62` *(et non* `0x22` *qui serait une simple addition).*

3. **Fonction avec clé multi-octets :**

```c
void xor_crypt(char *data, int len, const char *key) {
    int key_len = strlen(key);
    for (int i = 0; i < len; i++) {
        data[i] = data[i] ^ key[i % key_len];
    }
}
```

L'opérateur modulo (%) permet de "boucler" sur la clé : quand on atteint la fin de la clé, on recommence au début.

Exemple avec `"SECRET"` *(6 caractères) sur* `"FORMAT C: /Y"` *(12 caractères) :*

- *Position 0 :* 'F' $\oplus$ 'S'
- *Position 1 :* 'O' $\oplus$ 'E'
- ...
- *Position 6 :* ' ' $\oplus$ 'S' *(retour au début de la clé)*

4. **Point de vue antivirus :**

   (a) **Pourquoi la signature est inefficace ?**

   Chaque copie du virus utilise une clé différente, générée aléatoirement. Le code chiffré produit donc une séquence d'octets différente à chaque fois.

   Exemple avec le payload `"FORMAT C: /Y"` *:*

   - *Avec clé 0x42 :* `04 0D 10 0F 03 16 62 01 78 62 6D 1B`
   - *Avec clé 0x5A :* `1C 15 08 17 1B 0E 7A 19 60 7A 75 03`

   Impossible de créer une signature unique pour détecter toutes les variantes.

(b) ***Techniques alternatives de détection :***

- ***Émulation / Sandboxing :*** *Exécuter le code dans un environnement contrôlé pour observer son comportement après déchiffrement*
- ***Détection du décrypteur :*** *La routine de déchiffrement (partie non chiffrée) présente des patterns reconnaissables (boucle XOR)*
- ***Analyse heuristique :*** *Détecter les séquences de code caractéristiques du déchiffrement XOR*
- ***Analyse entropique :*** *Le code chiffré a une entropie élevée (distribution quasi-aléatoire des octets)*

Point clé à retenir

- *Le XOR est* ***involutif*** *: appliquer deux fois la même opération avec la même clé redonne la valeur originale ($A \oplus K \oplus K = A$).*
- *Le* ***polymorphisme*** *rend chaque copie du virus unique en binaire, empêchant la détection par signature.*
- *Avec une clé d'un seul octet, il n'y a que 255 variantes possibles (clé 0x00 = pas de chiffrement). Le brute-force est trivial.*
- *Le chiffrement de* ***Vernam*** *(One-Time Pad) est le seul système prouvé incassable, mais il nécessite une clé aussi longue que le message, aléatoire et à usage unique.*

Exercice 3 (Virus compagnon et détournement du PATH)**.** Un **virus compagnon** est un type de malware qui ne modifie pas les fichiers existants mais se place à côté d'eux pour être exécuté à leur place. Une technique courante consiste à exploiter la variable d'environnement `PATH` pour intercepter l'exécution de commandes système.

Principe de l'attaque : Lorsqu'un utilisateur tape une commande (par exemple `ls`), le système recherche l'exécutable dans les répertoires listés dans `PATH`, dans l'ordre. Si un attaquant place un faux `ls` dans un répertoire prioritaire, c'est ce programme malveillant qui sera exécuté.

On considère le programme C suivant, conçu pour remplacer la commande `ls` :

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    // Phase 1: Code malveillant (simulation)
    FILE *f = fopen("/tmp/log.txt", "a");
    if (f) {
        fprintf(f, "Commande interceptee: ls\n");
        fprintf(f, "Repertoire: %s\n", getcwd(NULL, 0));
        fclose(f);
    }

    // Phase 2: Execution de la vraie commande
    execvp("/bin/ls", argv);

    return 0;
}
```

1. Expliquer le fonctionnement de ce programme. Pourquoi l'utilisateur ne remarque-t-il pas que son système est compromis ?
2. Comment un attaquant pourrait-il installer ce virus compagnon sur un système Linux ? Proposer deux méthodes différentes.
3. Que se passe-t-il si l'attaquant oublie la ligne `execvp("/bin/ls", argv)` ? Quelles conséquences cela aurait-il sur la furtivité de l'attaque ?
4. Proposer deux contre-mesures que l'utilisateur ou l'administrateur système pourrait mettre en place pour se protéger contre ce type d'attaque.
5. **Bonus :** Modifier le programme pour qu'il enregistre également les arguments passés à la commande `ls`.

Rappel de cours

Résolution des commandes via PATH :
Lorsqu'un utilisateur tape une commande sans chemin absolu (ex : `ls` au lieu de `/bin/ls`), le shell recherche l'exécutable dans les répertoires listés dans la variable d'environnement `PATH`, **dans l'ordre**.

Exemple de PATH typique sous Linux :

```
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/sbin
```

Le shell cherche dans l'ordre : `/usr/local/bin`, puis `/usr/bin`, puis `/bin`, etc. Le **premier** exécutable trouvé est celui qui est lancé.

La fonction `execvp()` :

- **Remplace** le processus courant par un nouveau programme (le processus appelant n'existe plus)
- Le `v` signifie que les arguments sont passés sous forme de tableau (`argv`)
- Le `p` signifie que le programme est recherché dans le PATH (sauf si un chemin absolu est donné)
- Contrairement à `system()`, `execvp()` ne crée pas de sous-processus (pas de `fork` implicite) et ne passe pas par un shell intermédiaire

Différence `execvp()` vs `system()` :

	`execvp()`	`system()`
Processus	Remplace le processus	Crée un sous-processus
Retour	Ne retourne pas (si succès)	Retourne le code de sortie
Shell	Pas de shell intermédiaire	Passe par `/bin/sh`
Sécurité	Plus sûr (pas d'injection shell)	Vulnérable à l'injection

Correction 3. *1.* ***Fonctionnement du programme :***

Le programme s'exécute en deux phases :

Phase 1 – Code malveillant :

- *Ouvre le fichier `/tmp/log.txt` en mode ajout (`"a"`)*
- *Enregistre que la commande `ls` a été interceptée*
- *Enregistre le répertoire courant (espionnage de l'activité)*
- *Ferme le fichier*

Phase 2 – Couverture :

- *`execvp("/bin/ls", argv)` remplace le processus courant par le vrai `ls`*
- *Les arguments originaux sont transmis intégralement*
- *Le résultat affiché est identique à celui de la vraie commande*

Pourquoi c'est furtif : *L'utilisateur voit le résultat normal de `ls`. La phase malveillante est invisible car elle ne produit aucune sortie à l'écran.*

2. ***Méthodes d'installation :***

Méthode 1 : Modification du PATH utilisateur

```
# Dans ~/.bashrc ou ~/.profile
export PATH="$HOME/.malware:$PATH"
# Puis placer le faux ls dans ~/.malware/ls
```

Méthode 2 : Utilisation du répertoire courant

```
# Si PATH contient "." en premier
export PATH=".:$PATH"
# Placer le faux ls dans le repertoire de travail de la victime
```

Autres méthodes :

- *Exploiter une vulnérabilité pour écrire dans `/usr/local/bin` (souvent avant `/bin` dans PATH)*
- *Créer un alias dans `.bashrc` : `alias ls='/chemin/malware'`*

Pourquoi ne jamais mettre "." dans le PATH

Si le répertoire courant (`.`) est dans le PATH (surtout en première position), un attaquant peut placer un exécutable malveillant nommé `ls`, `cat`, ou `sudo` dans ***n'importe quel répertoire*** *visité par la victime (ex : `/tmp`, un dossier partagé). Il suffit que la victime exécute une commande courante dans ce répertoire pour lancer le malware.*
C'est pourquoi les distributions Linux modernes n'incluent ***jamais*** *`.` dans le PATH par défaut. Sous Windows, le répertoire courant est cherché en premier, ce qui a historiquement causé de nombreuses vulnérabilités (DLL hijacking, etc.).*

3. ***Sans la ligne execvp() :***

- *La vraie commande `ls` n'est jamais exécutée*
- *L'utilisateur ne voit aucune sortie (ou juste le code de retour 0)*
- *L'utilisateur remarque immédiatement que quelque chose ne va pas*
- *Il va investiguer et probablement découvrir le malware*

- *La **furtivité est totalement compromise***

C'est pourquoi la technique du virus compagnon nécessite toujours d'exécuter le programme original après le code malveillant.

4. ***Contre-mesures :***

   Contre-mesure 1 : Utiliser des chemins absolus

```
alias ls='/bin/ls'
alias cat='/bin/cat'
# Ou utiliser directement /bin/ls au lieu de ls
```

   Contre-mesure 2 : Sécuriser le PATH

   - *Ne jamais mettre "." dans PATH*
   - *Vérifier régulièrement le contenu de PATH*
   - *S'assurer que les répertoires prioritaires sont protégés en écriture*

   Autres contre-mesures :

   - *Activer SELinux ou AppArmor*
   - *Surveiller les modifications de `.bashrc` et `.profile`*
   - *Utiliser `type -a ls` pour voir tous les `ls` disponibles*

5. ***Bonus – Enregistrement des arguments :***

```
FILE *f = fopen("/tmp/log.txt", "a");
if (f) {
    fprintf(f, "Commande interceptee: ls\n");
    fprintf(f, "Repertoire: %s\n", getcwd(NULL, 0));
    fprintf(f, "Arguments (%d):\n", argc);
    for (int i = 0; i < argc; i++) {
        fprintf(f, "  argv[%d]: %s\n", i, argv[i]);
    }
    fclose(f);
}
```

Point clé à retenir

- *Un **virus compagnon** ne modifie aucun fichier existant : il se place "à côté" pour être exécuté à la place du programme légitime.*
- *L'ordre de recherche dans le **PATH** est critique : le premier répertoire a la priorité.*
- *La fonction `execvp()` remplace le processus courant, assurant la transparence pour l'utilisateur.*
- *Ne **jamais** inclure `.` (répertoire courant) dans le PATH.*
- *La commande `type -a <cmd>` permet de vérifier tous les emplacements d'une commande.*

Exercice 4 (Routine de recherche de fichiers cibles)**.** Avant de pouvoir infecter des fichiers, un virus doit d'abord les localiser sur le système. Cette phase de **recherche de cibles** est cruciale : elle doit être efficace pour maximiser la propagation tout en restant discrète pour éviter la détection.

On considère le programme C suivant qui parcourt récursivement un répertoire à la recherche de fichiers ayant une extension spécifique :

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>

void search_targets(const char *path, const char *extension) {
    DIR *dir = opendir(path);
    if (dir == NULL) return;

    struct dirent *entry;
    char fullpath[1024];

    while ((entry = readdir(dir)) != NULL) {
        // Ignorer . et ..
        if (strcmp(entry->d_name, ".") == 0 ||
            strcmp(entry->d_name, "..") == 0)
            continue;

        snprintf(fullpath, sizeof(fullpath), "%s/%s", path, entry->
    d_name);

        struct stat st;
        if (stat(fullpath, &st) == -1) continue;

        if (S_ISDIR(st.st_mode)) {
            // Recursion dans les sous-repertoires
            search_targets(fullpath, extension);
        } else if (S_ISREG(st.st_mode)) {
            // Verifier l'extension
            char *ext = strrchr(entry->d_name, '.');
            if (ext && strcmp(ext, extension) == 0) {
                printf("Cible trouvee: %s (%ld octets)\n",
                       fullpath, st.st_size);
            }
        }
    }
    closedir(dir);
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <repertoire> <extension>\n", argv[0]);
        return 1;
    }

    printf("Recherche de fichiers %s dans %s...\n", argv[2], argv[1])
    ;
```

```
    search_targets(argv[1], argv[2]);

    return 0;
}
```

1. Analyser le code et expliquer le rôle des fonctions `opendir()`, `readdir()`, `stat()` et `closedir()`.
2. Pourquoi est-il important d'ignorer les entrées "." et ".." lors du parcours d'un répertoire ? Que se passerait-il sinon ?
3. Modifier la fonction `search_targets()` pour qu'elle recherche les fichiers exécutables (ayant la permission d'exécution) au lieu de filtrer par extension.
4. Le parcours récursif peut être problématique sur de grands systèmes de fichiers. Proposer une modification pour limiter la profondeur de récursion à N niveaux maximum.
5. Du point de vue d'un antivirus, quels comportements suspects ce type de programme pourrait-il déclencher ? Comment un antivirus comportemental pourrait-il détecter cette activité ?

Rappel de cours

Parcours de systèmes de fichiers en C (POSIX) :
Sous Unix/Linux, un répertoire est un fichier spécial qui contient une liste d'entrées (nom → inode). Le parcours se fait via l'API POSIX : `opendir()`, `readdir()`, `closedir()`.

Entrées spéciales dans chaque répertoire :

- `.` (point) : référence au répertoire lui-même
- `..` (deux points) : référence au répertoire parent

Les macros de test de type de fichier (définies dans `<sys/stat.h>`) :

Macro	Signification
`S_ISREG(m)`	Fichier régulier
`S_ISDIR(m)`	Répertoire
`S_ISLNK(m)`	Lien symbolique
`S_ISCHR(m)`	Périphérique caractère
`S_ISBLK(m)`	Périphérique bloc
`S_ISFIFO(m)`	Pipe nommé (FIFO)
`S_ISSOCK(m)`	Socket

Bits de permission d'exécution :

Constante	Valeur octale	Signification
`S_IXUSR`	0100	Exécution par le propriétaire (`-x---`)
`S_IXGRP`	0010	Exécution par le groupe (`---x--`)
`S_IXOTH`	0001	Exécution par les autres (`----x`)

Rappel des permissions octales Unix :

Octal	Binaire	Permission
0	000	Aucune
1	001	Exécution (`x`)
2	010	Écriture (`w`)
3	011	Écriture + Exécution (`wx`)
4	100	Lecture (`r`)
5	101	Lecture + Exécution (`rx`)
6	110	Lecture + Écriture (`rw`)
7	111	Lecture + Écriture + Exécution (`rwx`)

Exemple : `chmod 755 fichier` → `rwxr-xr-x` (propriétaire : tout, groupe et autres : lecture + exécution).

Correction 4. *1. **Rôle des fonctions :***

- *`opendir(path)` : Ouvre un répertoire et retourne un pointeur `DIR*` permettant de le parcourir. Retourne `NULL` en cas d'erreur (répertoire inexistant, permissions insuff-*

isantes).

- `readdir(dir)` *: Lit l'entrée suivante du répertoire. Retourne un pointeur vers une structure* `struct dirent` *contenant le nom du fichier (*`d_name`*). Retourne* `NULL` *quand toutes les entrées ont été lues.*
- `stat(path, &st)` *: Obtient les métadonnées d'un fichier (taille, permissions, type, dates). Remplit la structure* `struct stat` *passée en paramètre. Retourne -1 en cas d'erreur.*
- `closedir(dir)` *: Ferme le répertoire et libère les ressources allouées par* `opendir()`*. Important pour éviter les fuites de descripteurs de fichiers.*

2. ***Importance d'ignorer "." et ".." :***

   - *"." représente le répertoire courant*
   - *".." représente le répertoire parent*

   Sans ce filtrage :

   - *Avec "." : Le programme entrerait dans le même répertoire indéfiniment →* ***boucle infinie***
   - *Avec ".." : Le programme remonterait vers la racine puis redescendrait → parcours infini de tout le système*
   - *Dans les deux cas :* ***stack overflow*** *dû à la récursion infinie*

   Attention

   L'oubli du filtrage de "." et ".." est une erreur de programmation classique lors du parcours récursif de répertoires. Le programme entre en récursion infinie et finit par un segmentation fault (dépassement de pile). C'est aussi un risque pour un virus qui pourrait s'auto-infecter indéfiniment ou planter avant d'avoir pu se propager.

3. ***Recherche par permissions d'exécution :***

```c
void search_targets(const char *path) {
    DIR *dir = opendir(path);
    if (dir == NULL) return;

    struct dirent *entry;
    char fullpath[1024];

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 ||
            strcmp(entry->d_name, "..") == 0)
            continue;

        snprintf(fullpath, sizeof(fullpath), "%s/%s", path, entry
    ->d_name);

        struct stat st;
        if (stat(fullpath, &st) == -1) continue;

        if (S_ISDIR(st.st_mode)) {
            search_targets(fullpath);
```

```
        } else if (S_ISREG(st.st_mode)) {
            // Verifier la permission d'execution (owner)
            if (st.st_mode & S_IXUSR) {
                printf("Executable trouve: %s (%ld octets)\n",
                        fullpath, st.st_size);
            }
        }
    }
    closedir(dir);
}
```

Note : *On peut aussi vérifier* `S_IXGRP` *(groupe) et* `S_IXOTH` *(autres) pour une recherche plus complète :*

```
if (st.st_mode & (S_IXUSR | S_IXGRP | S_IXOTH))
```

4. ***Limitation de la profondeur de récursion :***

```
void search_targets(const char *path, const char *ext,
                    int depth, int max_depth) {
    if (depth > max_depth) return;  // Limite atteinte

    DIR *dir = opendir(path);
    if (dir == NULL) return;

    struct dirent *entry;
    char fullpath[1024];

    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0 ||
            strcmp(entry->d_name, "..") == 0)
            continue;

        snprintf(fullpath, sizeof(fullpath), "%s/%s", path, entry
    ->d_name);

        struct stat st;
        if (stat(fullpath, &st) == -1) continue;

        if (S_ISDIR(st.st_mode)) {
            // Incrementer la profondeur pour les sous-
    repertoires
            search_targets(fullpath, ext, depth + 1, max_depth);
        } else if (S_ISREG(st.st_mode)) {
            char *e = strrchr(entry->d_name, '.');
            if (e && strcmp(e, ext) == 0) {
                printf("Cible: %s\n", fullpath);
            }
        }
    }
    closedir(dir);
}

// Appel initial avec depth=0
search_targets("/home", ".c", 0, 3);  // Max 3 niveaux
```

5. ***Détection par antivirus comportemental :***

   Comportements suspects détectables :

   - *Parcours rapide et massif de l'arborescence de fichiers*
   - *Accès en lecture à de nombreux fichiers exécutables*
   - *Pattern caractéristique : beaucoup d'`opendir`/`readdir`/`stat`*
   - *Accès à des répertoires sensibles (`/bin`, `/usr/bin`)*

   Techniques de détection :

   - ***Rate limiting :*** *Alerter si un processus ouvre trop de répertoires par seconde*
   - ***Analyse des patterns d'accès :*** *Détecter le parcours systématique par extension*
   - ***Surveillance des appels système :*** *Comptabiliser les appels à `stat()` sur des exécutables*
   - ***Honeypots :*** *Placer des fichiers "appâts" et alerter s'ils sont accédés*

Point clé à retenir

- *Le parcours récursif de répertoires utilise le triptyque `opendir()` / `readdir()` / `closedir()`.*
- ***Toujours** filtrer "`.`" et "`..`" pour éviter les boucles infinies.*
- *Les macros `S_ISDIR`, `S_ISREG` testent le type de fichier ; les constantes `S_IXUSR/S_IXGRP/S_IXOTH` testent les permissions.*
- *Un virus efficace limite sa profondeur de recherche pour rester discret et éviter les crashs.*

Exercice 5 (Infection par ajout de code (Appending))**.** L'**infection par ajout** (appending) est une technique classique où le virus ajoute son code à la fin d'un fichier cible. Pour éviter d'infecter plusieurs fois le même fichier (ce qui pourrait le corrompre ou le rendre suspect), le virus utilise un **marqueur d'infection**.

On considère le programme C suivant qui simule ce mécanisme sur des fichiers texte :

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MARKER "[[INFECTED]]"
#define PAYLOAD "\n--- MALICIOUS CODE SIMULATION ---\n"

int is_infected(const char *filename) {
    FILE *f = fopen(filename, "r");
    if (f == NULL) return -1;

    char buffer[1024];
    while (fgets(buffer, sizeof(buffer), f) != NULL) {
        if (strstr(buffer, MARKER) != NULL) {
            fclose(f);
            return 1;  // Deja infecte
```

```c
        }
    }
    fclose(f);
    return 0;  // Non infecte
}

int infect_file(const char *filename) {
    if (is_infected(filename) == 1) {
        printf("[SKIP] %s est deja infecte\n", filename);
        return 0;
    }

    FILE *f = fopen(filename, "a");
    if (f == NULL) {
        printf("[ERROR] Impossible d'ouvrir %s\n", filename);
        return -1;
    }

    fprintf(f, "%s", PAYLOAD);
    fprintf(f, "%s\n", MARKER);
    fclose(f);

    printf("[INFECTED] %s\n", filename);
    return 1;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <fichier1> [fichier2] ...\n", argv[0]);
        return 1;
    }

    for (int i = 1; i < argc; i++) {
        infect_file(argv[i]);
    }
    return 0;
}
```

1. Expliquer le rôle du marqueur d'infection (`MARKER`). Que se passerait-il si le virus n'utilisait pas de marqueur ?
2. Pourquoi le fichier est-il ouvert en mode "a" (append) dans la fonction `infect_file()` ? Quel serait l'effet d'utiliser le mode "w" à la place ?
3. La fonction `is_infected()` parcourt tout le fichier ligne par ligne. Proposer une méthode plus efficace.
4. Le marqueur `[[INFECTED]]` est facilement détectable par un antivirus. Proposer une modification pour rendre le marqueur moins visible.
5. Du point de vue d'un antivirus, quelles caractéristiques de ce programme pourraient être utilisées pour le détecter ?

Rappel de cours

Techniques d'infection de fichiers :
Un virus peut infecter un fichier exécutable de trois manières principales :

1. **Écrasement (Overwriting)** : Le virus remplace le début du fichier par son propre code. Simple mais **destructif** : le programme original est détruit et ne fonctionne plus. Détection immédiate par l'utilisateur.
2. **Ajout (Appending)** : Le virus ajoute son code à la **fin** du fichier et modifie le point d'entrée pour s'exécuter en premier, puis rend la main au programme original. Le fichier reste fonctionnel.
3. **Insertion au début (Prepending)** : Le virus insère son code au **début** du fichier et décale le code original. Plus complexe mais le virus s'exécute en premier naturellement.

Schéma textuel – infection par ajout :

```
AVANT infection :            APRES infection :
+--------------------+       +--------------------+
| Programme          |       | Programme          |
| original           |       | original           |
|                    |       |                    |
+--------------------+       +--------------------+
                             | Code du virus      |
                             | (payload)          |
                             +--------------------+
                             | Marqueur           |
                             | [[INFECTED]]       |
                             +--------------------+
```

Le programme original reste intact, le virus s'ajoute après. Le marqueur en fin de fichier permet d'éviter la réinfection.

Correction 5. 1. ***Rôle du marqueur d'infection :***

Le marqueur sert à identifier les fichiers déjà infectés pour ***éviter la réinfection****.*

Sans marqueur, problèmes :

- *Le fichier grossit à chaque exécution du virus (payload ajouté plusieurs fois)*
- *La taille du fichier devient anormalement grande → détection facile*
- *Le fichier peut devenir corrompu ou inutilisable*
- *Performances dégradées (fichiers de plus en plus longs)*
- *Risque de remplir le disque dur*

Le marqueur est placé à la fin du fichier infecté pour pouvoir être vérifié rapidement.

2. ***Mode "a" vs mode "w" :***

Tableau comparatif des modes d'ouverture de fichier en C :

Mode	*Lecture*	*Écriture*	*Position initiale*	*Fichier existant*
"r"	*Oui*	*Non*	*Début*	*Doit exister*
"w"	*Non*	*Oui*	*Début*	***Vidé / Créé***
"a"	*Non*	*Oui*	*Fin*	*Préservé / Créé*
"r+"	*Oui*	*Oui*	*Début*	*Doit exister*
"w+"	*Oui*	*Oui*	*Début*	***Vidé / Créé***
"a+"	*Oui*	*Oui*	*Fin*	*Préservé / Créé*

Mode "a" (append) :

- *Positionne le curseur à la fin du fichier*
- *Les écritures sont ajoutées après le contenu existant*
- *Le contenu original est* ***préservé***
- *Le programme/fichier infecté reste fonctionnel*

Mode "w" (write) :

- *Efface tout le contenu existant*
- *Le fichier est* ***vidé*** *puis réécrit depuis le début*
- *Le contenu original est* ***détruit***
- *Le programme infecté ne fonctionne plus du tout*

Un virus doit préserver la fonctionnalité du fichier infecté pour :

- *Rester furtif (l'utilisateur ne remarque rien)*
- *Permettre sa propagation (le programme doit s'exécuter)*

3. ***Optimisation de is_infected() :***

   Puisque le marqueur est toujours ajouté à la fin du fichier, on peut lire directement la fin avec `fseek()` *:*

```c
int is_infected(const char *filename) {
    FILE *f = fopen(filename, "r");
    if (f == NULL) return -1;

    // Se positionner a la fin moins la taille du marqueur
    int marker_len = strlen(MARKER) + 2;  // +2 pour \n eventuel

    if (fseek(f, -marker_len, SEEK_END) == 0) {
        char buffer[64];
        if (fread(buffer, 1, marker_len, f) > 0) {
            buffer[marker_len] = '\0';
            if (strstr(buffer, MARKER) != NULL) {
                fclose(f);
                return 1;  // Infecte
            }
        }
    }

```

```
    fclose(f);
    return 0;  // Non infecte
}
```

Explication de `fseek()` :

- *`fseek(f, offset, origin)` déplace le curseur de lecture/écriture dans le fichier*
- *`SEEK_SET` : depuis le **début** du fichier (offset ≥ 0)*
- *`SEEK_CUR` : depuis la **position courante** (offset positif ou négatif)*
- *`SEEK_END` : depuis la **fin** du fichier (offset généralement ≤ 0)*

Ici, `fseek(f, -marker_len, SEEK_END)` positionne le curseur à `marker_len` octets avant la fin du fichier, permettant de lire uniquement les derniers octets.

Avantages :

- *Complexité O(1) au lieu de O(n)*
- *Très rapide même pour de gros fichiers*
- *Moins d'accès disque*

4. ***Obfuscation du marqueur :***

Technique 1 : Chiffrement XOR

```
// Marqueur chiffre (cle = 0x42)
#define MARKER_ENC "\x0e\x0e\x09\x0c\x04\x07\x25\x07\x04\x06\x0f\
    x0f"
// Dechiffrer avant comparaison
```

Technique 2 : Hash du fichier

Au lieu d'un marqueur textuel, stocker un hash des premiers octets du fichier. Si le hash correspond, le fichier est infecté.

Technique 3 : Marqueur dans les métadonnées

Utiliser les attributs étendus du fichier (xattr) ou modifier la date de modification de manière caractéristique.

Technique 4 : Caractères non-imprimables

```
#define MARKER "\x01\x02\x03\x04\x05\x06\x07\x08"
```

5. ***Techniques de détection antivirus :***

- ***Signature du marqueur :*** *Rechercher la chaîne `[[INFECTED]]` dans les fichiers*
- ***Surveillance des accès fichiers :*** *Détecter les patterns d'ouverture en mode append sur de nombreux fichiers*
- ***Intégrité des exécutables :*** *Comparer les hash des fichiers avec une base de référence (modification = alerte)*
- ***Analyse de la croissance :*** *Alerter si la taille d'un exécutable augmente de manière suspecte*
- ***Analyse comportementale :*** *Détecter un programme qui lit puis écrit dans de nombreux fichiers exécutables*

- ***Analyse statique :*** *Rechercher les patterns de code caractéristiques (fopen mode "a" + fprintf sur des fichiers variables)*

Point clé à retenir

- *Le* ***marqueur d'infection*** *empêche la réinfection et garantit que les fichiers restent fonctionnels.*
- *Le mode* `"a"` *(append) est essentiel : il préserve le contenu original du fichier, contrairement au mode* `"w"` *qui le détruit.*
- `fseek(f, -n, SEEK_END)` *permet de lire les n derniers octets d'un fichier en O(1), optimisation cruciale pour la vérification d'infection.*
- *Les techniques d'obfuscation du marqueur (XOR, hash, métadonnées) compliquent la détection mais n'empêchent pas l'analyse comportementale.*